

Exercise set 3

Shrinkage methods

Ridge regression and LASSO are called *shrinkage methods* since the regression coefficients are shrunk towards zero.

- OLS: Minimize $RSS(\beta) = \sum_{i=1}^n (y_i - \mathbf{x}_i' \beta)^2$
- Ridge regression: Minimize $RSS(\beta) + \lambda \sum_{k=1}^p \beta_k^2$
- LASSO: Minimize $RSS(\beta) + \lambda \sum_{k=1}^p |\beta_k|$

Task 1

Consider the dataset [College](#). You shall predict the number of applications, **Apps**. Restrict the dataset by

```
require(ISLR2)
xdata=subset(College,select=c("Apps","Accept","Top10perc","Expend"))
```

Use the help-function for [College](#) to get a description of the variables that you have chosen.

- a) Compare the estimated coefficients for OLS, ridge regression and LASSO in a regression of **Apps** on **Accept**, **Top10perc** and **Expend**. Try with different values for λ , $\lambda = 0.01, 0.1, 1, 10, 100, 1000$ and perhaps a few others. An example with $\lambda = 100$. Ridge regression and LASSO can be fitted with the `glmnet`-function. As an example, to obtain the coefficients from Ridge regression use

```
library(glmnet)
beta_ridge=glmnet(as.matrix(xdata[,-1]),as.matrix(xdata[,1]),lambda=100,alpha=0)$beta
```

- b) Use the `cv.glmnet` function to find the optimal λ for both ridge regression and LASSO. Explain what the function is doing.

```
cvridge=cv.glmnet(as.matrix(xdata[,-1]),as.matrix(xdata[,1]),alpha=0)
lambda_ridge=cvridge$lambda.min
lambda_ridge
```

```
FALSE [1] 364.8993
```

- c) Use the optimal λ 's and reestimate the ridge regression and the LASSO. Compare the coefficients with the OLS-estimates.
- d) Evaluate the three predictions by `testMSE` using the validation set approach. Example with ridge regression.

```
n=nrow(xdata)
ind=sample(1:n,size=floor(n/2))
train=xdata[ind,]
test=xdata[-ind,]
Xtrain=as.matrix(xdata[ind,-1])
Xtest=as.matrix(xdata[-ind,-1])
ytrain=as.matrix(xdata[ind,1])
ytest=as.matrix(xdata[-ind,1])
ridge=glmnet(Xtrain,ytrain,alpha=0,lambda=lambda_ridge)
pred_ridge=predict(ridge,newx=Xtest)
```

```
pred_ols=predict(lm(Apps~.,data=train),newdata=test)
mean((ytest-pred_ols)^2)
```

```
FALSE [1] 1399610
```

```
mean((ytest-pred_ols)^2)
```

```
FALSE [1] 1293261
```

OLS works better than Ridge regression in this case to predict Apps. Try with LASSO also but remember to use the optimal λ for that method then (and set $\alpha = 1$).

Non-linear models

- In *polynomial regression* Y is regressed on $1, X, \dots, X^K$.
- In *piecewise constant regression* Y is regressed on indicator variables $I(X < c_1), I(c_1 \leq X < c_2), \dots, I(c_{K-1} \leq X < c_K), I(X \geq c_K)$.
- In the *basis function approach* Y is regressed on a number of transformations of X , $b_0(X), b_1(X), b_2(X), \dots, b_K(X)$.
- A *cubic spline basis* is defined by

$$\begin{aligned} & (b_0(x), b_1(x), b_2(x), b_3(x), b_4(x), \dots, b_{K+3}(x)) \\ & = (1, x, x^2, x^3, h(x, \xi_1), \dots, h(x, \xi_K)) \end{aligned}$$

where

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise} \end{cases}$$

- In *regression splines* Y is regressed on a cubic spline basis.

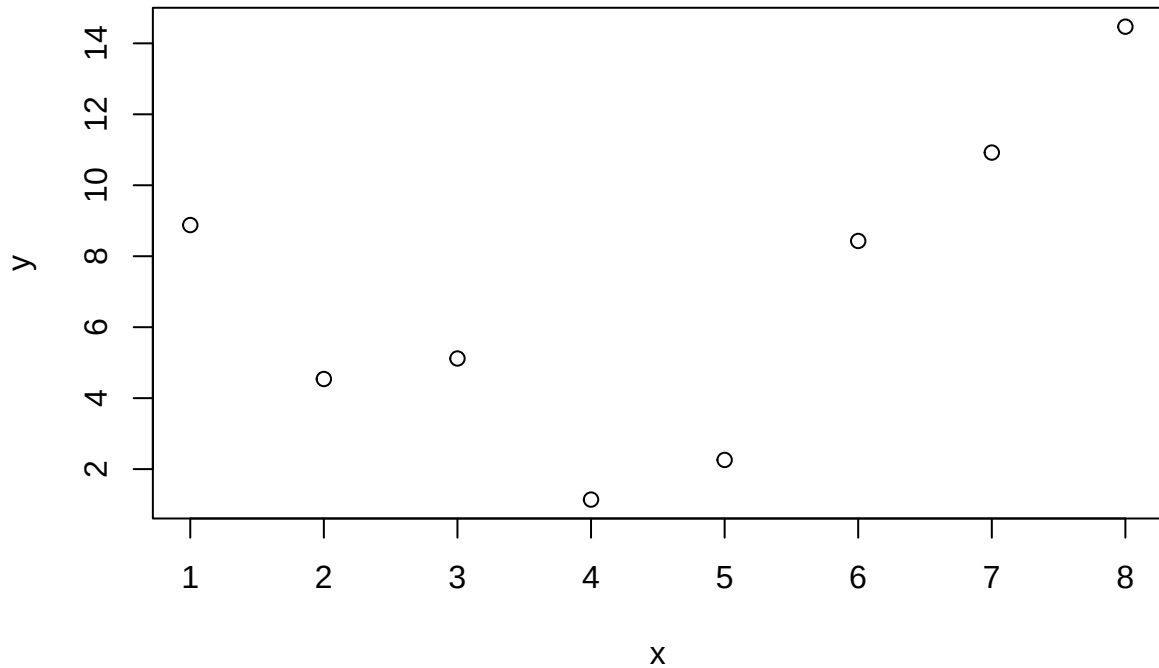
Task 2

Consider the data set

x	y
1	8.88
2	4.54
3	5.12
4	1.14
5	2.26
6	8.43
7	10.92
8	14.47

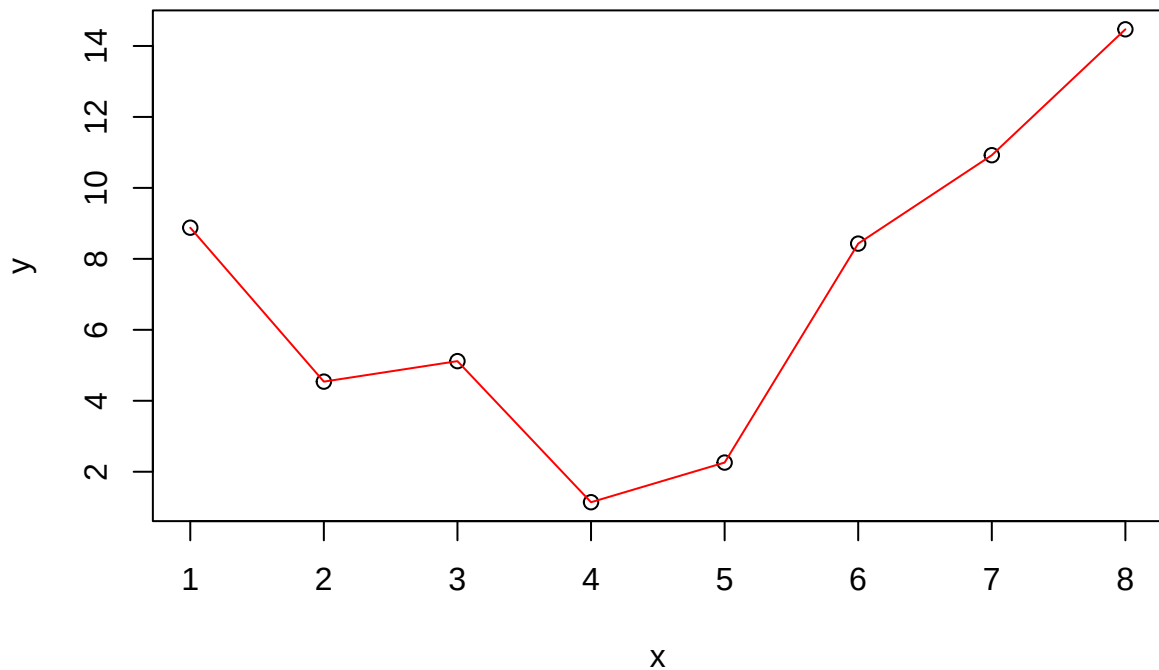
- a) Plot y against x .

```
plot(x,y)
```



- b) Fit a polynomial regression of order $K = 7$ to the data. Plot the predictions for $x = 1, \dots, 8$ on top of the data.

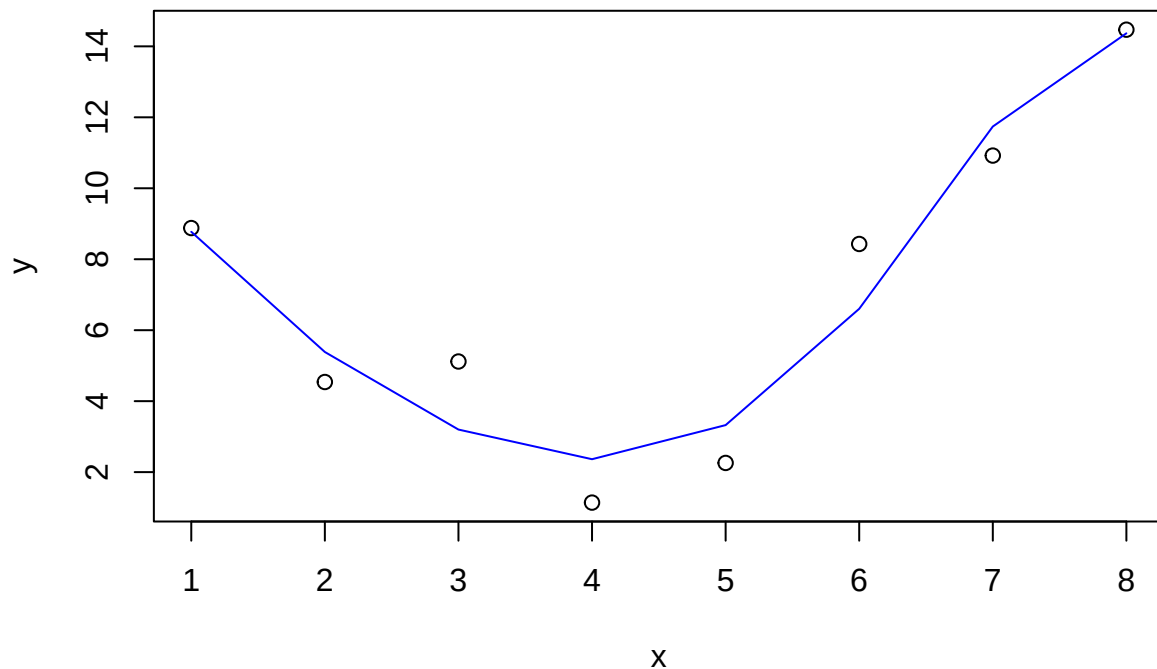
```
reg1=lm(y~x+I(x^2)+I(x^3)+I(x^4)+I(x^5)+I(x^6)+I(x^7))
plot(x,y)
lines(x,predict(reg1),col="red")
```



perfect fit.

- c) Fit a regression spline using the `lm`-function with two knots, $\xi_1 = 3$ and $\xi_2 = 6$. Plot the predictions obtained on top of the data. Do you expect predictions from this model to be better or worse than the prediction in b) for a new x -value? Make sure that you understand what the code is doing.

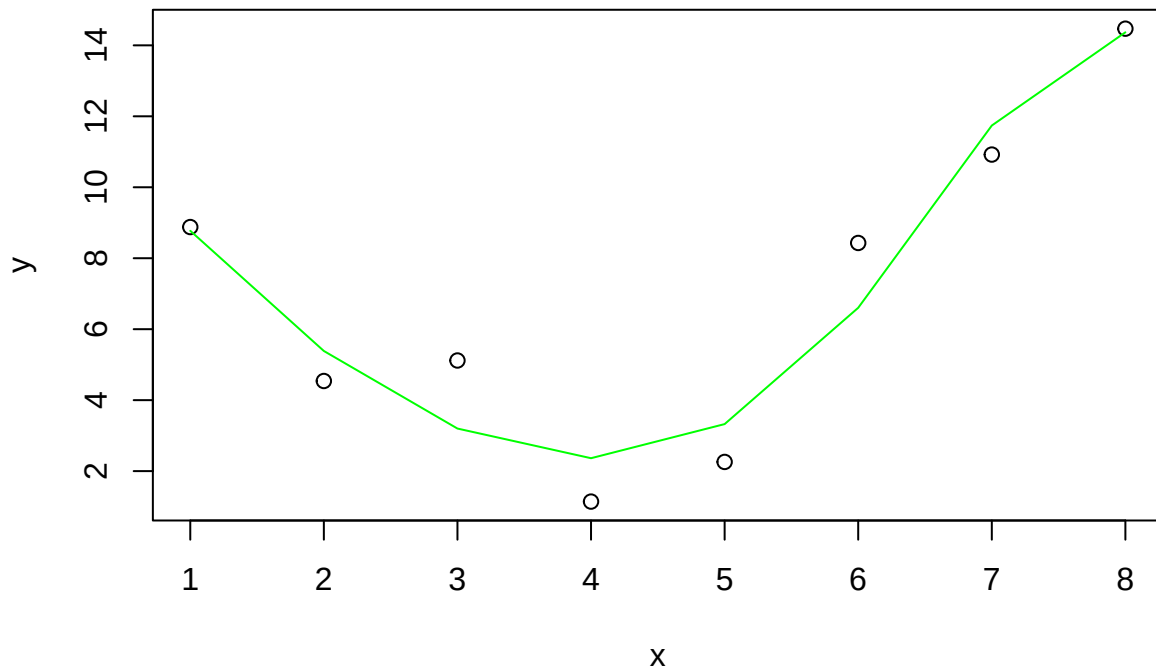
```
h=function(x,xi) pmax((x-xi)^3,0)
rs=lm(y~x+I(x^2)+I(x^3)+h(x,3)+h(x,6))
plot(x,y)
lines(x,predict(rs),col="blue")
```



It is difficult to know if the regression spline will do well for a new observation. However the model in c) with a perfect fit seems unlikely to perform well. Imagine, e.g., an observation outside of the interval $[1, 8]$.

- d) Use the R-function `bs` in the `splines`-package to produce the same predictions as in c).

```
library(splines)
rs2=lm(y~bs(x,knots=c(3,6)))
plot(x,y)
lines(x,predict(rs2),col="green")
```



Also print the predicted values to see that they are identical.

```
predict(rs)
```

```
FALSE      1      2      3      4      5      6      7      8
FALSE 8.773272 5.385859 3.199513 2.363725 3.324674 6.602712 11.740205 14.367581
```

```
predict(rs2)
```

```
FALSE      1      2      3      4      5      6      7      8
FALSE 8.773272 5.385859 3.199513 2.363725 3.324674 6.602712 11.740205 14.367581
```

- e) Investigate different placements of the knots and determine which of them are best using the leave-one-out cross-validation.

```
require(splines)
testMSE=function(xi1,xi2,x,y)
{
  n=length(x)
  SE=matrix(0,n,1)
  xy=data.frame(x=x,y=y)
  for(i in 1:n)
  {
    rs=lm(y~x+I(x^2)+I(x^3)+h(x,xi1)+h(x,xi2),data=xy[-i,])
    pred=predict(rs,newdata=xy[i,])
    SE[i]=(y[i]-pred)^2
  }
  MSE=mean(SE)
  return(MSE)
}
testMSE(3,6,x,y)
```

```
FALSE [1] 2044.783
```

```
testMSE(2.5,6.5,x,y)
```

```
FALSE [1] 41097.5
```

```
testMSE(3.5,5.5,x,y)
```

```
FALSE [1] 376.9983
```

Not a formal optimization but the knots 3.5 and 5.5 are the best we have found so far.

An extension of KNN

- In *K nearest neighbors* (KNN) we predict y_0 by

$$\hat{f}(x_0) = \frac{1}{K} \sum_{k=1}^K y_{(k)},$$

$y_{(k)}$ is the y -value corresponding to the x -value which is the k 'th closest to x_0 .

- An extension of KNN is to predict y_0 by

$$\hat{f}(x_0) = \frac{1}{n} \sum_{i=1}^n w_i y_i,$$

i.e., we predict y_0 by a weighted average of observed all y_i 's, $i = 1, 2, \dots, n$.

Task 3

- a) Two needed properties of the weights w_i , $i = 1, 2, \dots, n$ are that they should sum to one and that a large w_i should correspond to a small deviation between x_i and x_0 . Explain why we need these properties?

We would like the weights to sum to one so that the predictions maintain the same magnitude as the original data. A small deviation between x_i and x_0 corresponding to a large weight for y_i means that we give a larger weight to observations “similar” (in terms of x) to the one we would like to predict.

- b) One way, different from the way we have done it previously, to implement KNN in R is the following

```
weight=function(x0,x,K)
{
  n=length(x)
  w=matrix(0,n,1)
  dist=abs(x-x0)
  odist=order(dist)
  w[odist[1:K]]=1/K
  return(w)
}
knn2=function(x0,x,y,K=3)
{
  w=weight(x0,x,K=K)
  ypred=sum(w*y)
  return(ypred)
}
```

Make sure that you understand why this give the same result as

```
knn1=function(x0,x,y,K=3)
{
  dist=abs(x-x0)
  odist=order(dist)
  y0=mean(y[odist[1:K]])
  return(y0)
}
```

Experiment with a small dataset, e.g.

```
set.seed(12)
x=seq(-50,50,length=100)
y=1-0.05*x-0.02*x^2+0.003*x^3+50*rnorm(100)
```

to see and understand the equality of knn1 and knn2.

By using the simulated x and y we can make predictions for all observed x's with both knn1 and knn2

```
ypred1=apply(as.matrix(x),1,knn1,x=x,y=y)
ypred2=apply(as.matrix(x),1,knn2,x=x,y=y)
head(ypred1)
```

```
FALSE [1] -411.9287 -411.9287 -379.2998 -416.5158 -383.6476 -352.9941
```

```
head(ypred2)
```

```
FALSE [1] -411.9287 -411.9287 -379.2998 -416.5158 -383.6476 -352.9941
```

KNN can be written as a special case of the more general method of predicting y_0 by a weighted average of all observations. The weight function **weight** is, for KNN, equal to $1/K$, for the K observations with x -values closest to x_0 and zero otherwise. `knn2` is doing this in two steps by first computing the weights in one function and then call that function from `knn2`. `knn1`, on the other hand is not explicitly computing the weights but simply identifies observations close to each other in the x -dimension and then averages the corresponding y 's.

c) If we change the weight function to

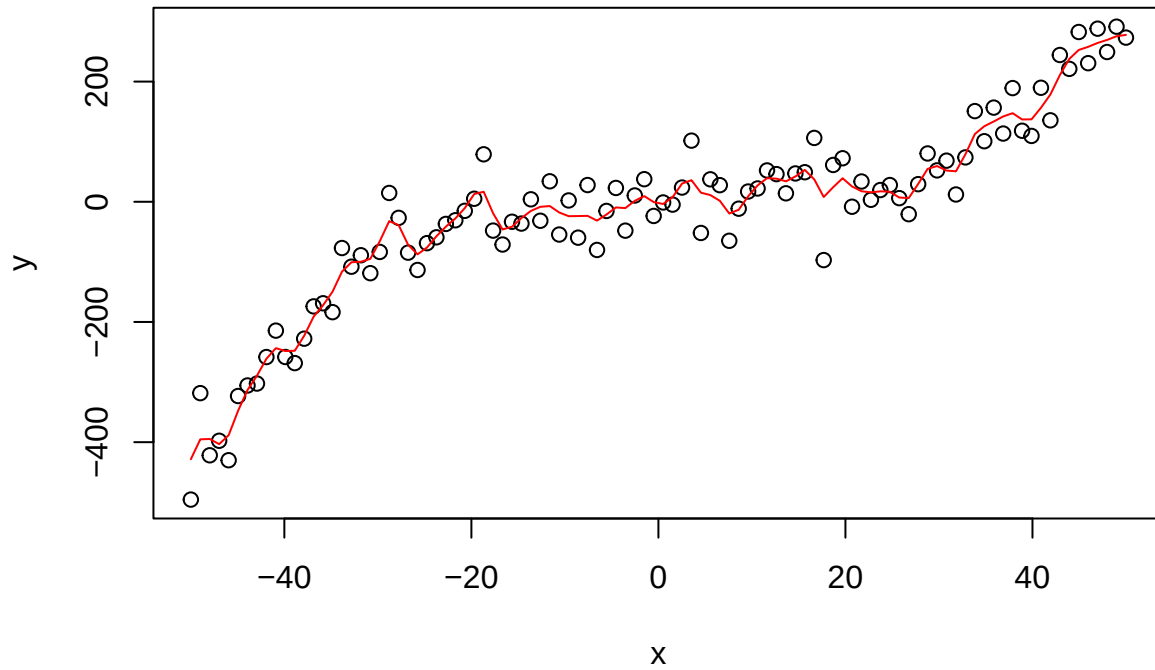
$$w_i = K(x_i - x_0) = \frac{\phi\left(\frac{x_i - x_0}{h}\right)}{\sum_{i=1}^n \phi\left(\frac{x_i - x_0}{h}\right)}$$

where ϕ is the standard normal density we get the Nadaraya-Watson estimator. In R we can implement this by

```
weight=function(x0,x,h=0.2) dnorm((x-x0)/h)/sum(dnorm((x-x0)/h))
```

Modify the function `knn2` above so it becomes the Nadaraya-Watson estimator. Apply it to the data simulated in b). Plot it on top of the data. Try with a few different values of h .

```
nw=function(x0,x,y,h=0.2)
{
  w=weight(x0,x,h=h)
  ypred=sum(w*y)
  return(ypred)
}
ypred3=apply(as.matrix(x),1,nw,x=x,y=y,h=1)
plot(x,y)
lines(x,ypred3,col="red")
```



d) What is the meaning of h in c)?

The shape of the weight-function, the kernel, is the density of the normal distribution. h is measuring the scale of this function and is related to the variance. It determines how “local” the estimator should be. If h is very small only observations very close to x_0 will have an impact; for very large h , the prediction will be the average of all y ’s.